

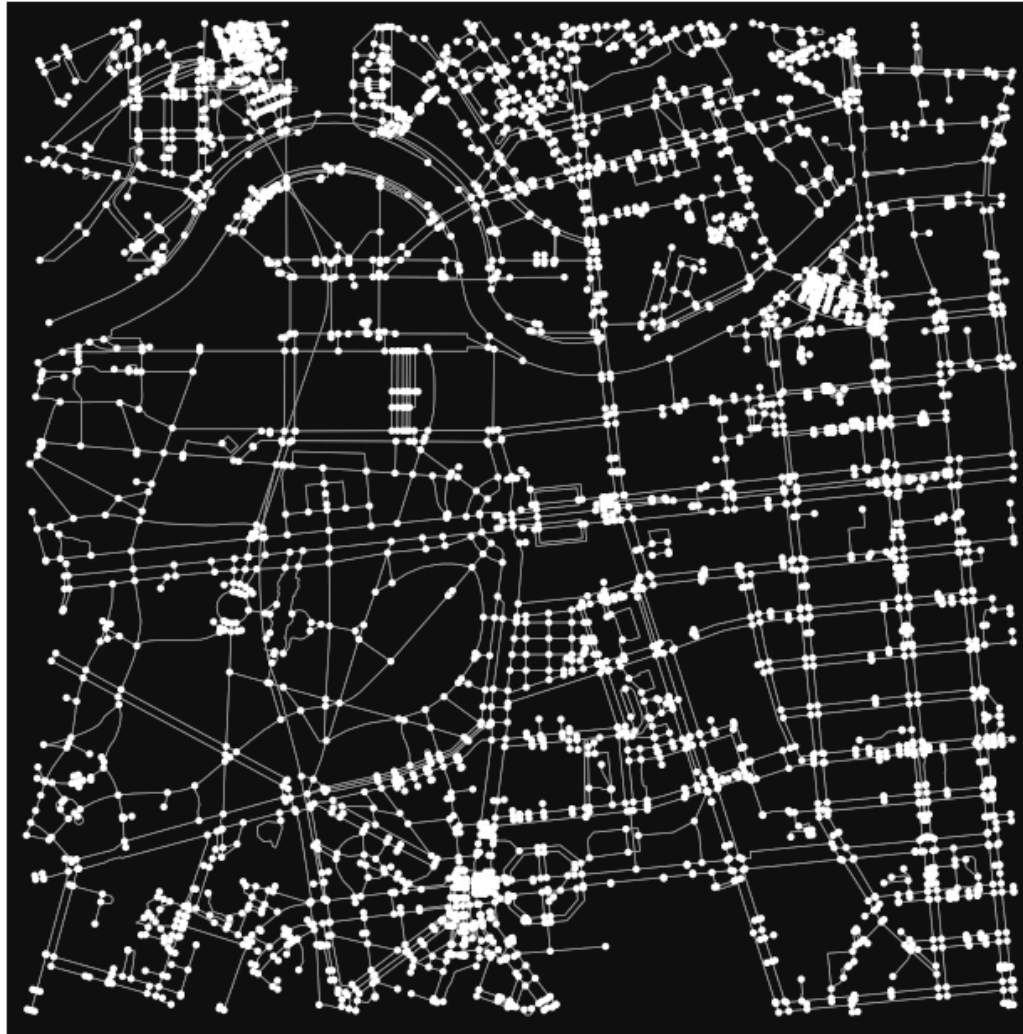
```
!pip install geopandas osmnx networkx matplotlib contextily folium
```

```
import geopandas as gpd
import osmnx as ox
import networkx as nx
import matplotlib.pyplot as plt
import folium
from shapely.geometry import Point
```

```
#setting a central point in Berlin (Branden Gate),Germany
latitude = 52.5163
longitude = 13.3777

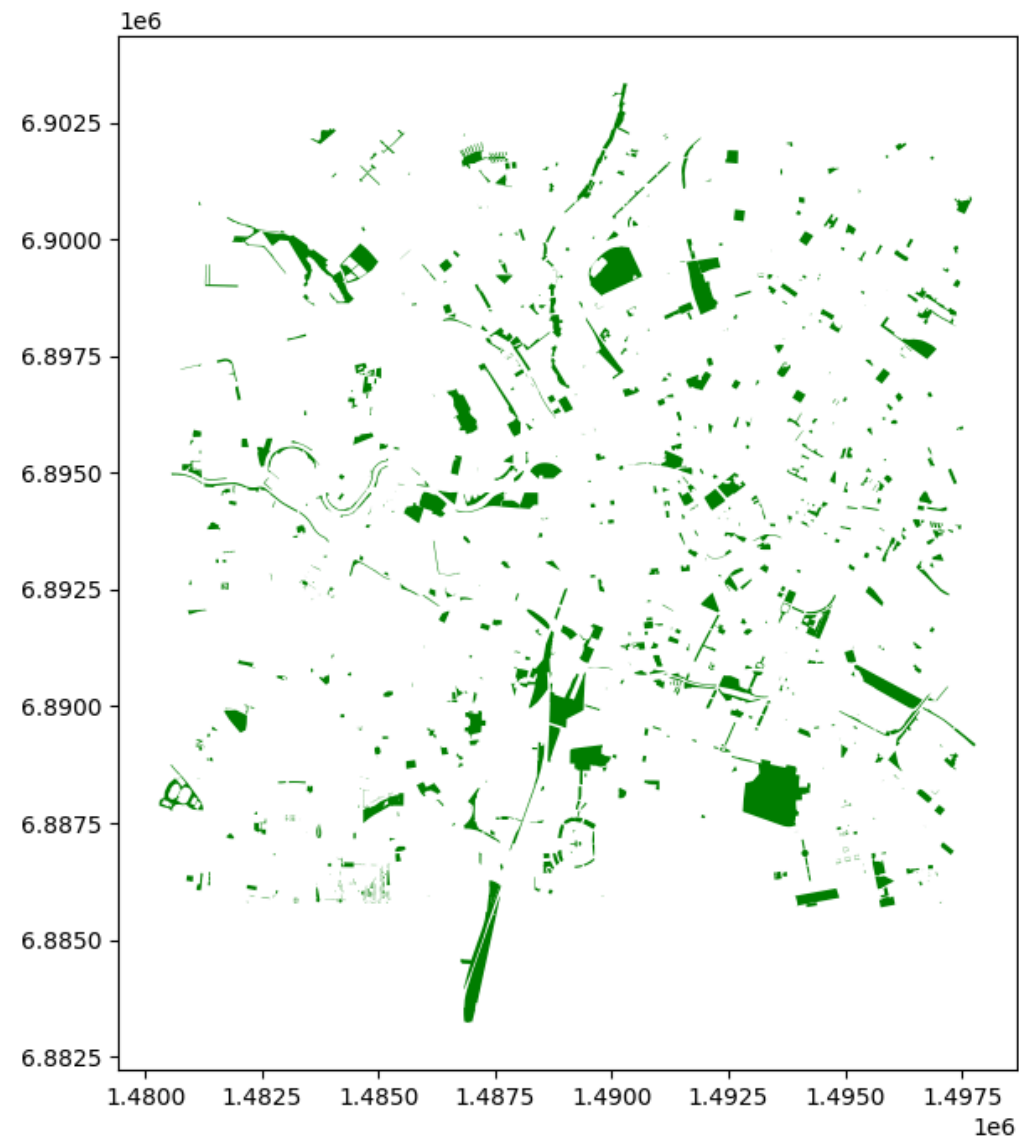
#creating a bounding graph from a 1km radius
G = ox.graph_from_point((latitude,longitude), dist=1000,network_type='walk')
```

```
#plotting the street network
fig, ax = ox.plot_graph(G,node_size=10, edge_linewidth=0.5)
```



```
tags = {'leisure': ['park', 'garden', 'recreational_ground', 'nature_reserve']}  
#getting the green areas (polygon geometries)  
green_spaces = ox.features_from_point((latitude, longitude), tags=tags, dist=5000)  
  
#filtering polygons only (extracted from 'streetmap')  
green_spaces = green_spaces[green_spaces.geometry.type.isin(['Polygon',  
                                                             'Multipolygon'])]  
  
green_spaces = green_spaces.to_crs(epsg=3857)  
green_spaces.plot(color='green', figsize=(8, 8))
```

<Axes: >



```
import random
from shapely.geometry import Point, Polygon
from shapely.ops import unary_union

#creating convex hull or bounding polygon
```

```

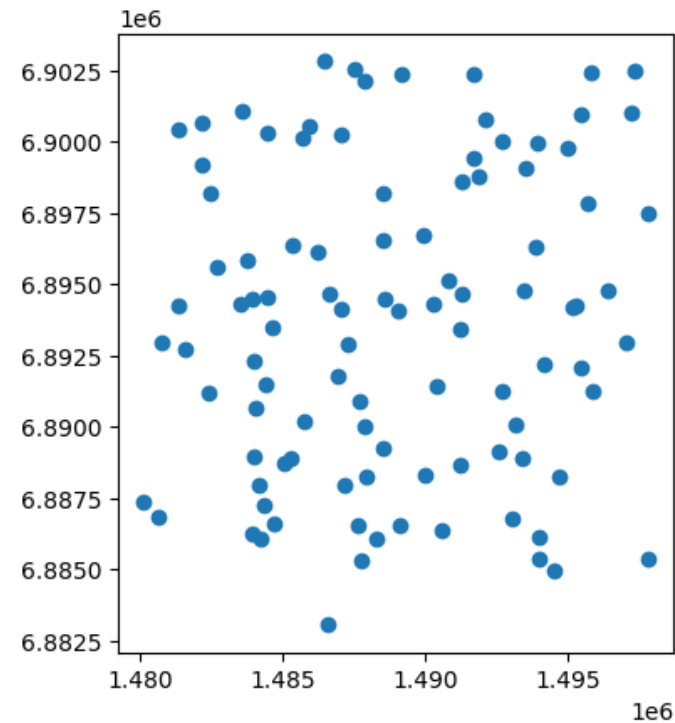
area_polygon = unary_union(green_spaces.buffer(1000).geometry)

#generate random residential points within polygon
res_points = []
while len(res_points)<100 :
    x=random.uniform(area_polygon.bounds[0],area_polygon.bounds[2])
    y=random.uniform(area_polygon.bounds[1],area_polygon.bounds[3])
    point=Point(x,y)
    if area_polygon.contains(point):
        res_points.append(point)

res_gdf = gpd.GeoDataFrame(geometry=res_points,crs="EPSG:3857")
res_gdf.plot()

```

<Axes: >



```

#convert back to WGS84
res_gdf = res_gdf.to_crs(epsg=4326)

#getingg nearest nodes on the walking network

```

```

res_nodes = [ox.distance.nearest_nodes(G,point.x,point.y)
for point in res_gdf.geometry]

park_centroids = green_spaces.geometry.centroid
park_nodes = [ox.distance.nearest_nodes(G,pt.x,pt.y) for pt in park_centroids]

```

```

#building a dataframe of the shortest distances
distances = []

unique_park_nodes = list(set(park_nodes)) # Ensure unique park nodes

for res_node in res_nodes:
    # Compute shortest paths from the residential node to all reachable nodes in the graph
    distances_from_res_node = nx.single_source_dijkstra_path_length(G, source=res_node, weight='length')

    min_distance_to_park = float('inf')
    for park_node in unique_park_nodes:
        if park_node in distances_from_res_node: # Check if the park node is reachable
            min_distance_to_park = min(min_distance_to_park, distances_from_res_node[park_node])

    if min_distance_to_park == float('inf'):
        # If no park is reachable from this residential node, append a placeholder
        # (e.g., None or a very large number)
        distances.append(None)
    else:
        distances.append(min_distance_to_park)

res_gdf['green_space_distance_m'] = distances
res_gdf['green_space_distance_km'] = res_gdf['green_space_distance_m'] / 1000

```

```

#creating a folium map
m = folium.Map(location=[latitude, longitude], zoom_start=13)

# Convert green_spaces to EPSG:4326 for Folium
green_spaces_4326 = green_spaces.to_crs(epsg=4326)

#adding the green spaces
for _,row in green_spaces_4326.iterrows():
    folium.GeoJson(row['geometry'],
                    style_function=lambda x: {'color': 'green'}).add_to(m)

#adding residential points
for _,row in res_gdf.iterrows():
    folium.CircleMarker(location=[row['geometry'].v, row['geometry'].x],

```


m



```
res_gdf.to_file("residential_green_access.geojson", driver="GeoJSON")
```

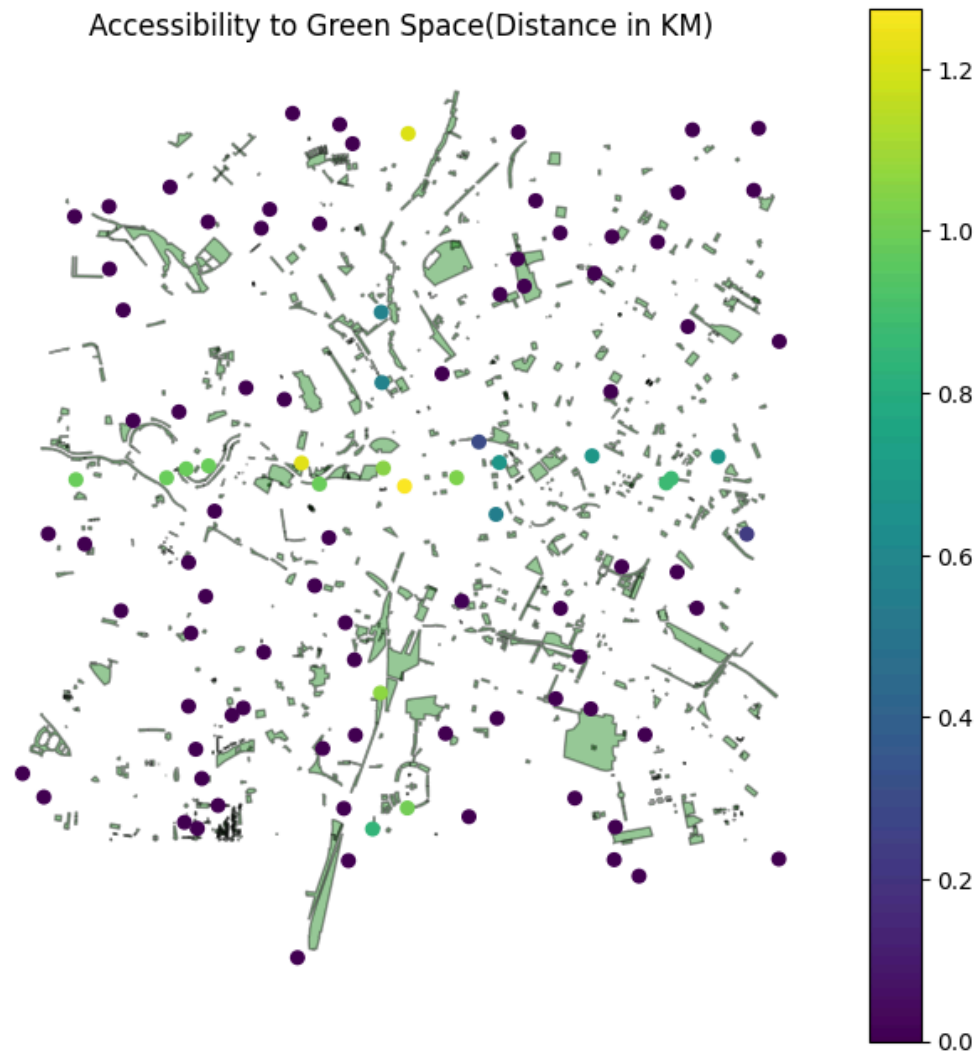
✓ Analysis

```
import matplotlib.pyplot as plt

res_gdf_plot = res_gdf.to_crs(epsg=3857)

fig, ax = plt.subplots(figsize=(8, 8))
green_spaces.to_crs(epsg=3857).plot(ax=ax, color='green', alpha=0.4, edgecolor='black')
res_gdf_plot.plot(ax=ax, column='green_space_distance_km',
                  cmap='viridis', legend=True, markersize=30)

plt.title("Accessibility to Green Space(Distance in KM)")
plt.axis('off')
plt.show()
```



- Darker colors (closer to purple) indicate residential points that are closer to a green space.
- Lighter colors (closer to yellow) indicate residential points that are further away from a green space.

```
from matplotlib import color_sequences
import folium
from branca.colormap import linear
```



```
colour_scale=linear.viridis.scale(
    res_gdf['green_space_distance_km'].min(),
    res_gdf['green_space_distance_km'].max()
)

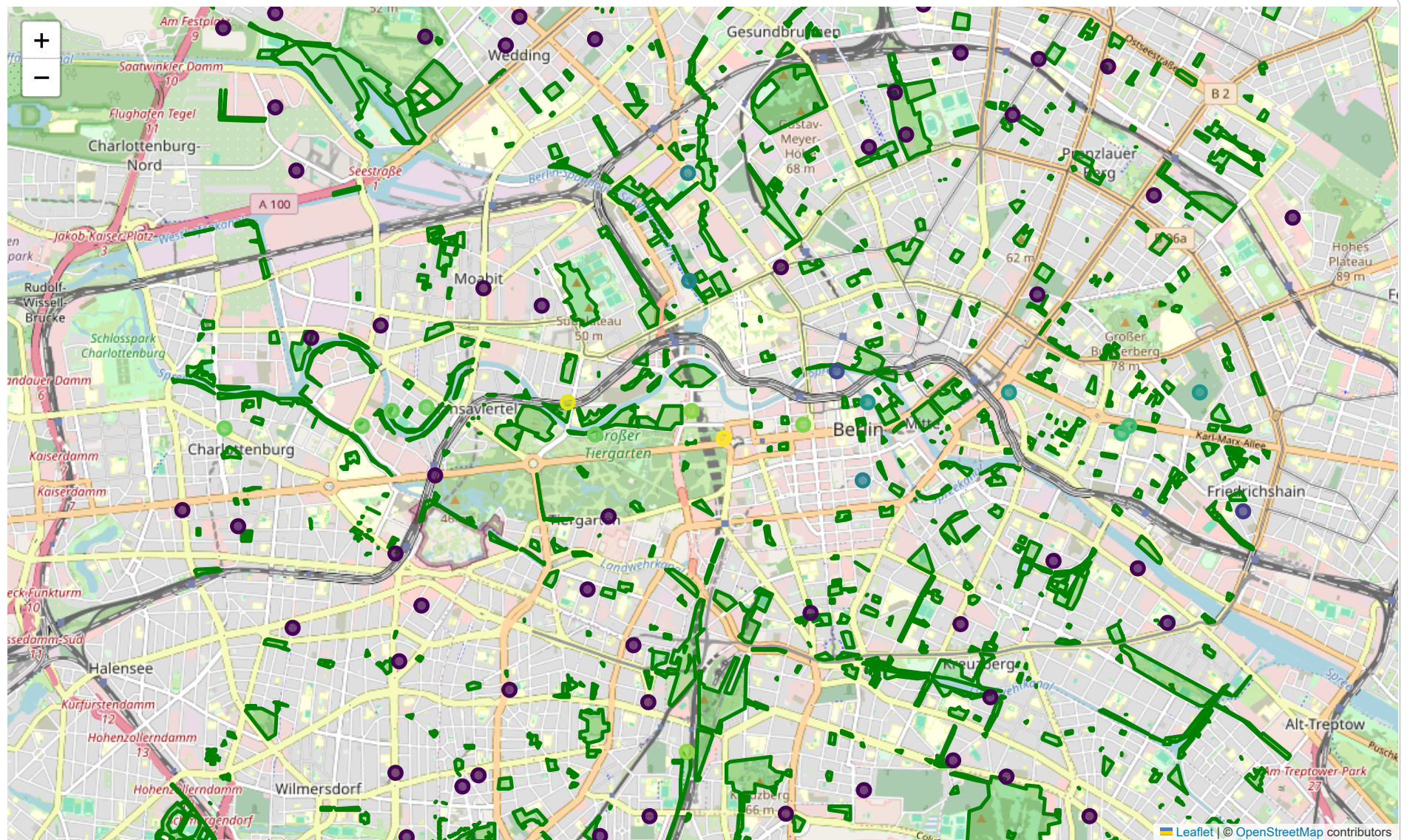
m = folium.Map(location=[latitude, longitude], zoom_start=13)

# Convert green_spaces to EPSG:4326 for Folium
green_spaces_4326 = green_spaces.to_crs(epsg=4326)

#adding the green spaces
for _,row in green_spaces_4326.iterrows():
    folium.GeoJson(row['geometry'],
        style_function=lambda x: {'color': 'green'}).add_to(m)

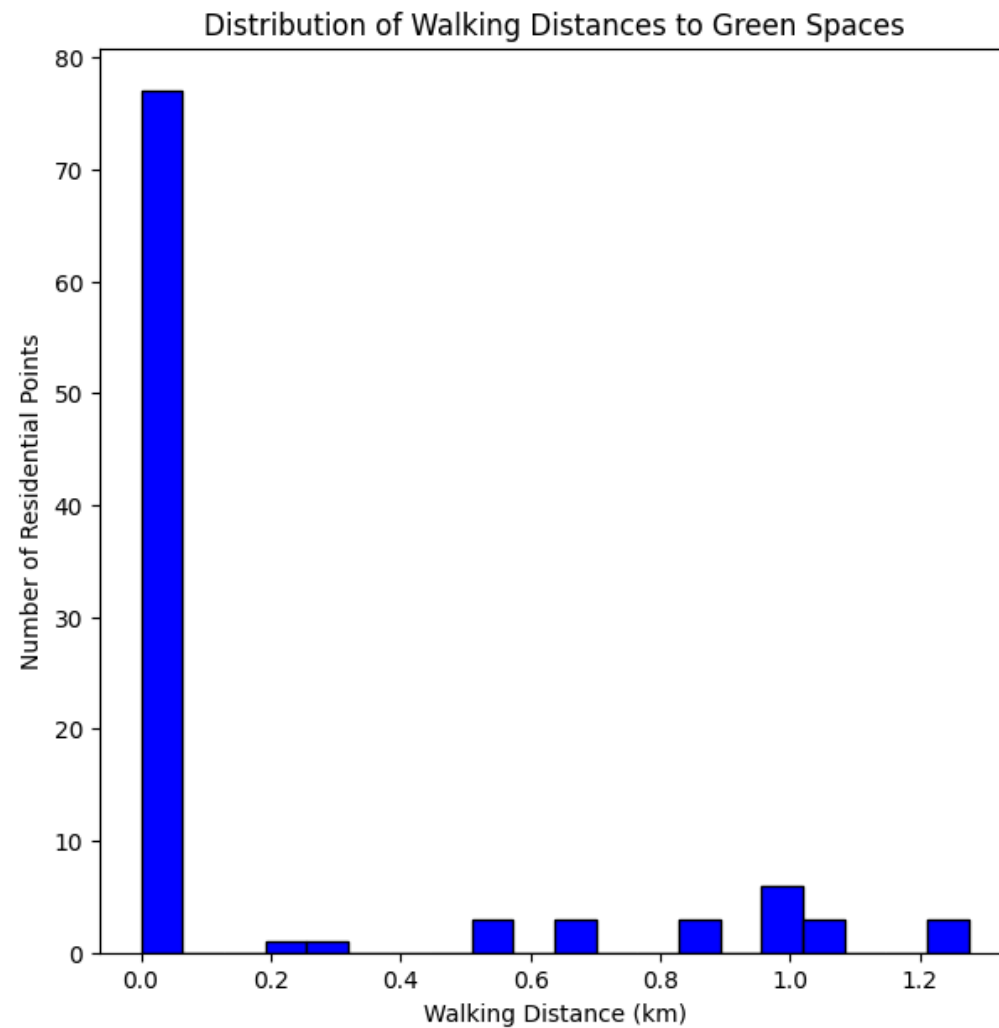
#adding residential points WITH color-coded distances
for _,row in res_gdf.iterrows():
    folium.CircleMarker(location=[row['geometry'].y, row['geometry'].x],
        radius=5,
        color=colour_scale(row['green_space_distance_km']),
        fill=True, fill_opacity=0.7,
        popup=f"Distance: {row['green_space_distance_km']:.2f} km").add_to(m)

m
```



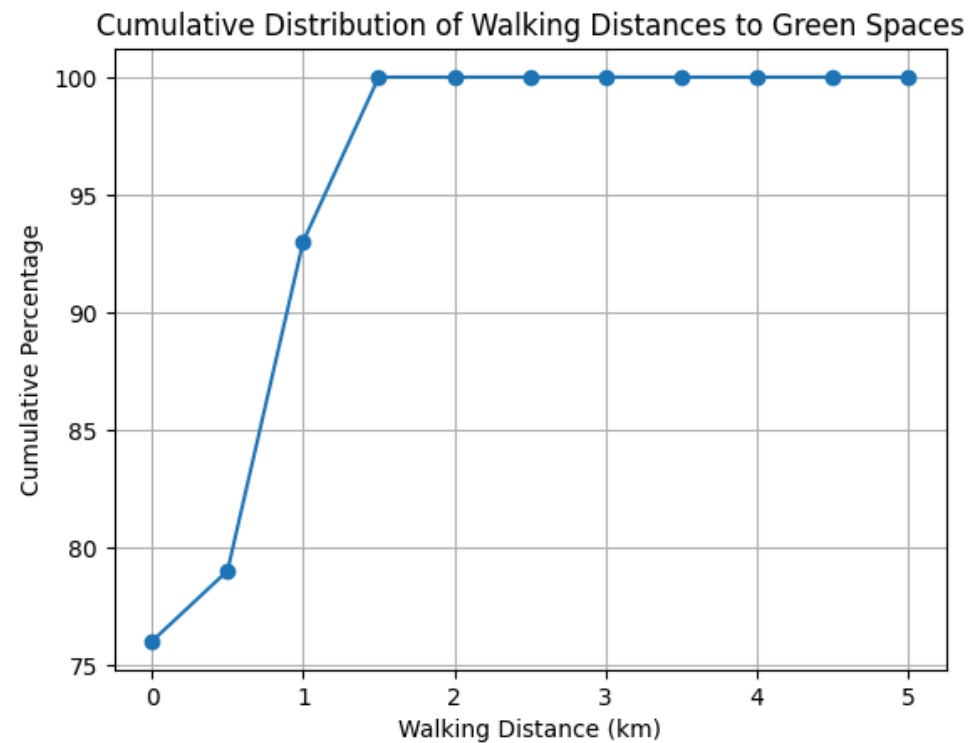
```
from IPython.core.pylabtools import figsize
#plotting an graph for distribution of walkin distances
res_gdf['green_space_distance_km'].plot.hist(bins=20,color='blue',
                                              edgecolor='black',
                                              figsize=(7,7)
                                              )
```

```
plt.title("Distribution of Walking Distances to Green Spaces")  
plt.xlabel("Walking Distance (km)")  
plt.ylabel("Number of Residential Points")  
plt.show()
```



```
import numpy as np  
  
thresholds = np.arange(0,5.5,0.5)  
cumulative = [(res_gdf['green_space_distance_km']<=t).mean()*100 for t in thresholds]
```

```
#plot
import matplotlib.pyplot as plt
plt.plot(thresholds,cumulative, marker='o', linestyle='-')
plt.title("Cumulative Distribution of Walking Distances to Green Spaces")
plt.xlabel("Walking Distance (km)")
plt.ylabel("Cumulative Percentage")
plt.grid(True)
plt.show()
```



```
print("Accessibility Summary:")
print(f"Min Distance: {res_gdf['green_space_distance_km'].min():.2f} km")
print(f"Max Distance: {res_gdf['green_space_distance_km'].max():.2f} km")
print(f"Mean Distance: {res_gdf['green_space_distance_km'].mean():.2f} km")
print(f"Median Distance: {res_gdf['green_space_distance_km'].median():.2f} km")
```

Accessibility Summary:
Min Distance: 0.00 km
Max Distance: 1.28 km

Mean Distance: 0.20 km
Median Distance: 0.00 km

- **Min Distance: 0.00 km**

Explanation: This means that at least one, and likely several, residential points are located directly within or immediately adjacent to a green space. This indicates excellent, direct access for some residents. Evaluation: This is ideal, as it represents immediate access to green amenities.

- **Max Distance: 1.21 km**

Explanation: The furthest any residential point is from a green space on the walking network is 1.21 kilometers. This represents the longest walking distance a resident would need to travel to reach the nearest green area. Evaluation: A maximum distance of 1.21 km is quite good. It suggests that even the least accessible residential points are still within a reasonable walking distance (typically, a